MINI PROJECT REPORT

ON

TOURIST GUIDE – A PROGRESSIVE WEB APPLICATION FOR EXPLORING TOURIST DESTINATIONS IN INDIA

Under the guidance of

Ms. Thrisha
Lecturer
Department of Computer Science

Submitted by

Mohammed Sahran Sattar
24BCA359

BACHELOR OF COMPUTER APPLICATIONS

(Artificial Intelligence, Cloud Computing and DevOps)



YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE & MANAGEMENT

(A constituent unit of Yenepoya Deemed to be University) Deralakatte, Mangaluru

575018, Karnataka, India

OCTOBER 2026

CERTIFICATE

This is to certify that the mini project work entitled "**Tourist Guide – A Progressive Web Application for Exploring Tourist Destinations in India**" has been successfully carried out by **Mohammed Sahran Sattar (24BCA359)**, student of **3rd Year BCA (Artificial Intelligence, Cloud Computing and DevOps)**, under the supervision and guidance of **Ms. Thrisha, Lecturer, Department of Computer Science, Yenepoya Institute of Arts, Science, Commerce and Management**. This dissertation is submitted in partial fulfillment of **Continuous Internal Assessment** for the **V semester Software Engineering and Progressive Web Apps Course** during academic year 2026-27.

Course Instructor: Ms. Thrisha Chairperson

PRINCIPAL
Dr. Jeevan Raj
The Yenepoya Institute of Arts,
Science, Commerce and Management
(Deemed To be university)

Submitted for the CIA

Place: Mangaluru
Date: October 2026

DECLARATION

I, **Mohammed Sahran Sattar**, a student of BCA (Artificial Intelligence, Cloud Computing and DevOps) at Yenepoya Institute of Arts, Science, Commerce and Management, Kulur, Mangaluru, affiliated with Yenepoya (Deemed to be University), hereby declare that this Mini Project report titled "Tourist Guide – A Progressive Web Application for Exploring Tourist Destinations in India" is a genuine and original record of the work undertaken by me as part of my academic curriculum.

This record comprises the Continuous Internal Assessment activities completed up to the date of submission, including Structured Software Document and UML modelling course-related learning activities as prescribed by the course instructor. The work presented in this record is my own and has been carried out in accordance with the academic guidelines and assessment requirements of the University.

I express my sincere gratitude to the Course Instructor for providing continuous guidance, encouragement, and constructive feedback throughout the assessment process. I also extend my appreciation to the Head of the Department and the faculty members of Yenepoya Institute of Arts, Science, Commerce and Management for creating a supportive learning environment that has enabled me to successfully complete the assessment activities undertaken so far.

I hereby affirm that the information furnished in this record is true to the best of my knowledge. I understand that any instance of plagiarism, fabrication, or academic dishonesty may result in appropriate disciplinary action as per the regulations of Yenepoya (**Deemed to be University**).

Intern Signature:

Course Instructor Signature

Name: Ms. Thrisha

Institute: Yenepoya Institute of Arts, Science, Commerce and Management

Date:

Head of the Department (HOD) Signature:

Name: Dr. Rathnakar Shetty P

Head of Department of Computer Science

Institute: Yenepoya Institute of Arts, Science, Commerce and Management

Date:

ACKNOWLEDGEMENT

I sincerely express my gratitude to Yenepoya Institute of Arts, Science, Commerce and Management, affiliated with Yenepoya (Deemed to be University), Mangaluru, for providing me with the opportunity to undertake this project as part of my academic curriculum.

I extend my heartfelt thanks to Dr. Jeevan Raj, Principal, for his constant encouragement and for fostering an academic environment that promotes experiential learning and research. I also express my sincere appreciation to Dr. Rathnakar Shetty P, Head of the Department, for his continuous support, guidance, and motivation throughout the course of this project.

I am deeply grateful to Ms. Thrisha, Course Instructor, for providing valuable guidance, constructive suggestions, and continuous encouragement during the planning, execution, and completion of this mini project. Her expertise and academic insights have greatly contributed to the successful completion of this work.

I extend my heartfelt thanks to all the faculty members of the Department of Computer Science for their valuable suggestions, encouragement, and support throughout the preparation of this report.

Lastly, I express my sincere gratitude to my parents, family members, friends, and classmates for their unwavering encouragement, moral support, and motivation throughout the completion of this project. Their constant support has been a source of inspiration and strength during this academic endeavour.

Name: Mohammed Sahran Sattar
Reg No: 24BCA359
Date: 01/10/2026

ABSTRACT

Travellers who plan a visit to a destination in India usually gather information from several different websites. The details they need, such as the location of a place, a description of it, its visiting timings, its entry fee and the category to which it belongs, are scattered across these sources and are difficult to compare. When the network is weak or unavailable during the journey, even the pages that were read earlier can no longer be opened.

Tourist Guide is a web-based **Progressive Web Application** developed to address this problem by providing a single, organised platform on which tourist destinations in India can be discovered and explored. A registered user can log in, browse the available tourist places, search for a place, filter the list by category, open a detailed page for any place, and save places as favourites for quick access later. An Admin Module is provided through which an administrator logs in separately, adds, edits and deletes tourist place information, and views the registered users and system statistics.

The application is developed using HTML5, CSS3 and JavaScript for the front end and Python Flask for the back end. Data is stored in a relational database: MySQL is used as the primary database through the PyMySQL driver, and the application falls back automatically to SQLite when MySQL is not available. A Web App Manifest, a Service Worker and the Cache Storage API turn the application into a Progressive Web Application that can be installed on a device and whose cached pages and static resources remain accessible when the device is offline.

This report documents the software engineering life cycle of the proposed system. It presents the problem definition and objectives, the software requirements specification, the layered architecture, workflow, data flow, database and interaction models that describe the design of the system, the modular breakdown adopted for implementation, and a set of hypothetical test cases with defined success and failure criteria.

TABLE OF CONTENTS

Sl.no	Topic	Page No.
1.	CHAPTER 1: INTRODUCTION	8
1.1	Overview	8
1.2	Objectives	9
1.3	Purpose	9
1.4	Overview of Tools and Technologies Used (Software & Hardware)	10
1.5	Problem Statement	12
2.	CHAPTER 2: SOFTWARE REQUIREMENTS SPECIFICATION	14
2.1	User Requirements	14
2.2	System Requirements	15
2.3	Design and Implementation Constraints	16
2.4	Specific Requirements	17
2.4.1	Interface Requirements	17
2.4.2	Functional Requirements	18
2.4.3	Non-Functional Requirements	19
3.	CHAPTER 3: SYSTEM DESIGN AND ANALYSIS	21
3.1	System Architecture Design	21
3.2	Workflow Diagrams	22
3.3	Database Design	26
3.4	Data Flow Diagrams	29
3.5	Sequence Diagrams	30
3.6	Security and Session Design	32
4.	CHAPTER 4: IMPLEMENTATION	33
4.1	Development Approach	33
4.2	Module Description	35
4.3	User Interface Screenshots	37
5.	CHAPTER 5: TESTING	40
5.1	Testing Objectives	40
5.2	Hypothetical Test Cases (Input, Expected Output, Success-Failure Criteria)	41
6.	CHAPTER 6: CONCLUSION	44
7.	CHAPTER 7: FUTURE ENHANCEMENTS	45
8.	BIBLIOGRAPHY	47

LIST OF TABLES

Table No.	Description	Page No.
1	Technology Stack of the Tourist Guide Application	10
2	Access Rights of Visitors, Users and Administrators	15
3	Minimum Hardware Requirements	15
4	Software Requirements	16
5	Structure of the users Table	27
6	Structure of the tourist_places Table	28
7	Structure of the favorites Table	28
8	Structure of the admins Table	28
9	Summary of System Modules	36
10	Traceability of Functional Requirements to Modules	37
11	Hypothetical Test Cases	42

LIST OF FIGURES

Fig no.	Description	Page No.
1	System Architecture Diagram	21
2	User Workflow Diagram	23
3	Admin Workflow Diagram	24
4	Offline Request Handling Workflow	25
5	Use Case Diagram	26
6	Entity-Relationship Diagram	27
7	Data Flow Diagram – Level 0 (Context Diagram)	29
8	Data Flow Diagram – Level 1 (User Processes)	29
9	Data Flow Diagram – Level 1 (Admin Processes)	30
10	Sequence Diagram – User Login	30
11	Sequence Diagram – Search, View Details and Add to Favorites	31
12	Sequence Diagram – Admin Adds a Tourist Place	31
13	Registration and Login Page of the Tourist Guide Application	38
14	Home Dashboard	38
15	Tourist Places Page with Search and Category Filter	38
16	Place Details Page	39
17	Favorites Page	39
18	Admin Dashboard	39

INTRODUCTION

1.1. Overview

Tourist Guide – A Progressive Web Application for Exploring Tourist Destinations in India is a web-based platform designed to help users discover and explore tourist destinations in India in a simple and organised manner. It brings together the activities that a traveller normally performs across several separate websites: finding places worth visiting, reading about them in detail, and keeping a short list of the places that are of particular interest.

A user of the system begins by registering an account and logging in. After a successful login the Home Dashboard is presented as the starting point of the application. From there the user can browse the list of tourist places, search for places and filter the list by category. Selecting a place opens a details page that shows the information stored for that destination, namely its location, description, visiting timings, entry fee and category. The user may add a place to a list of favourites, remove it again later, and view all saved places on a favourites page whenever quick access is needed. When the session is complete the user logs out.

The platform recognises two categories of users. The User is a traveller who browses, searches and saves places. The Admin is responsible for the content of the platform. The administrator logs in through a separate administrative login, views the Admin Dashboard, adds new tourist places, edits or deletes existing ones, and views the registered users together with system statistics. Because tourist destination information is managed entirely through this Admin Module, the content that users see can be kept current without changing the application itself.

The front end of the application is developed using HTML5, CSS3 and JavaScript. HTML5 provides the structure of the pages, CSS3 controls the presentation and the responsive layout, and JavaScript supplies the interactive behaviour. The back end is developed in Python 3 using the Flask framework, which handles routing and application logic. Data is stored in a relational database: MySQL is supported as the primary database, and the application switches automatically to a SQLite database if MySQL cannot be reached.

In addition to these technologies the application uses the features that distinguish a Progressive Web Application from an ordinary website. A Web App Manifest allows the application to be installed on a device and launched like a native application. A Service Worker, together with the Cache Storage API, keeps pages and static resources in the browser so that they remain accessible when the device is offline. Live database updates, however, still require an internet connection.

The scope of the present report is the design and structured documentation of the system. It covers the analysis of requirements, the architecture and the workflow, data flow, database and interaction models of the proposed solution, the modules into which the system is divided, and the strategy that will be followed to test it.

1.2. Objectives

The main objectives of the Tourist Guide application are listed below.

- To provide a single, centralised web platform on which tourist destinations in India can be browsed and explored.
- To provide user registration, login and logout so that each user has a personal account.
- To provide a Home Dashboard that acts as the starting point of the application after login.
- To allow tourist places to be browsed as a list and to be searched.
- To allow the list of tourist places to be filtered by category.
- To present detailed information for each destination, including its location, description, visiting timings, entry fee and category.
- To allow a user to add places to a favourites list, remove them, and view the saved places.
- To provide a separate administrator login and an Admin Dashboard.
- To allow the administrator to add, edit and delete tourist place information so that the content stays up to date.
- To allow the administrator to view the registered users and system statistics.
- To store application data in a relational database, using MySQL as the primary database with an automatic fallback to SQLite.
- To implement the application as a Progressive Web Application, using a Web App Manifest, a Service Worker and the Cache Storage API, so that it can be installed and its cached resources remain accessible offline.
- To design a responsive interface that is compatible with desktop, tablet and mobile devices.
- To document the software engineering life cycle of the project, from requirements analysis through design and implementation to testing.

1.3. Purpose

The purpose of the Tourist Guide application is to reduce the effort a traveller spends in finding reliable and organised information about tourist destinations in India. In ordinary practice a traveller opens several websites in turn to learn where a place is, what it offers, when it can be visited and how much it costs to enter. Each website presents this information in a different form, and the traveller must compare and remember the details manually.

This project addresses that situation by providing one place in which the information is kept in a uniform structure. Every destination is described by the same set of details, so places can be

compared easily, searched for, and filtered by category. A favourites list allows a traveller to keep the places of interest together and return to them quickly without repeating the search.

A second purpose is to keep the application usable when the network is not. During travel the connection is often weak or absent. Implementing the system as a Progressive Web Application means that its pages and static resources that have been cached remain accessible without a connection, and that the application can be installed on a device and opened in the same way as a native application.

A third purpose is to give the destination information a proper owner. Through the Admin Module an administrator maintains the tourist places stored in the system, adding new destinations and correcting or removing existing ones, so that users always see the information that the administrator has approved.

From an academic point of view, the project also provides an opportunity to work through a complete software development life cycle in a structured manner. It requires the documentation of requirements, the design of a layered architecture and a relational database, the definition of the modules of a web application, and the preparation of a test strategy. It also provides practical experience with a Python Flask back end, MySQL and SQLite databases, and the technologies that define a Progressive Web Application, such as the Web App Manifest, the Service Worker and the Cache Storage API.

Scope of the Project

The scope of the system is the presentation and management of tourist destination information for India. It serves two actors, the User and the Admin, and it covers registration and login, browsing, searching and filtering, place details, favourites, the administrative management of places, and the offline and installation capabilities of a Progressive Web Application. Capabilities such as maps, reviews, notifications and itinerary planning are outside the present scope and are recorded as future enhancements in Chapter 7.

1.4. Overview of Tools and Technologies Used (Software & Hardware)

This section describes the technologies selected for the development of the system and the hardware on which the system will be developed and tested. The technology stack is summarised in Table 1.

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Python 3, Flask
Database	MySQL with automatic SQLite fallback
PWA	Web App Manifest, Service Worker, Cache Storage API
Dependencies	PyMySQL, Werkzeug

Table 1: Technology Stack of the Tourist Guide Application

Software Technologies

HTML5: HyperText Markup Language defines the structure and content of every page of the application. It provides the elements from which the registration and login pages, the Home Dashboard, the places list, the place details page, the favourites page and the administrative screens are built, including headings, forms, input fields, buttons, lists and navigation regions.

CSS3: Cascading Style Sheets control the appearance of the application. CSS3 is used to define the colour scheme, typography and spacing, to arrange the interface using Flexbox and Grid layouts, and to apply media queries so that the layout adapts to desktop, tablet and mobile screen widths.

JavaScript: JavaScript supplies the behaviour of the pages in the browser. It validates form input before submission, responds to user actions such as searching and filtering, and registers the Service Worker that provides offline support.

Python 3 and Flask: Flask is a lightweight Python web framework. It is used for backend routing and application logic: it receives the requests sent from the browser, applies the rules of the application, reads from and writes to the database, and returns the pages and data that the user interface displays. Its session support is used to remember whether a user or an administrator is logged in.

PyMySQL and Werkzeug: PyMySQL is the Python driver through which the application communicates with the MySQL database. Werkzeug is the utility library on which Flask is built; its security helpers are used so that passwords are stored as hashes instead of plain text.

MySQL with SQLite Fallback: MySQL is the primary relational database, holding users, administrators, tourist places and favourites. If a connection to MySQL cannot be established, the application switches automatically to a SQLite database file, so that the application continues to work in environments where a MySQL server is not available.

Web App Manifest, Service Worker and Cache Storage API: The manifest is a JSON file declaring the name, icons, theme colour, display mode and start URL of the application, which allows the browser to offer it for installation. The Service Worker is a script that the browser runs in the background; it intercepts network requests and uses the Cache Storage API to serve cached pages and static resources when the device is offline.

Development Tools: A code editor such as Visual Studio Code is used for development. The developer tools built into modern browsers are used to inspect the layout, debug JavaScript, examine the Service Worker and the contents of Cache Storage, and simulate offline conditions.

Hardware Requirements

The application can be developed and tested on a standard computer or laptop with the following configuration.

- Processor: Dual-core processor or higher
- RAM: 4 GB or above
- Storage: Sufficient free space for the development tools, the project files and the database
- Display: Standard monitor or laptop display
- Network: Internet connection for installing dependencies and for testing live database updates
- Mobile device: A smartphone or tablet is useful for verifying the responsive layout and the installation behaviour of the application

1.5. Problem Statement

Tourists often need to search through different websites to find information about destinations. The details they require, including the location of a place, a description of it, its visiting timings, its entry fee and the category to which it belongs, are rarely found together. One website may describe a fort in detail but omit the entry fee, while another gives the timings but does not say where the place lies in relation to other attractions.

Comparing destinations is a particular difficulty. Because every source presents its information differently, a traveller who wants to know which places of a certain kind are available must open several pages and collect the details by hand. There is also no convenient way to remember the places that seemed interesting, so the same searches are often repeated.

Access to online information can also become difficult when network connectivity is weak or unavailable during travel. A page that was read at home can no longer be opened on the road, precisely when the traveller needs it most.

There is therefore a need for a simple, centralised platform in which tourist destinations can be browsed, searched and filtered, in which the details of each place are presented in a consistent form, in which places of interest can be saved as favourites, and in which previously loaded pages and static resources remain accessible without a network connection. The proposed Tourist Guide application addresses this need, with a separate Admin Module through which the destination information is maintained.

Proposed Solution

Tourist Guide provides a simple, centralised platform where users can browse, search and view tourist destinations. The destination details are stored once, in a relational database, and are presented to every user in the same form. Users can save destinations as favourites, while the Progressive Web Application features allow cached pages and static resources to remain accessible when the application is offline.

Key Modules of the System

The solution is organised into the following key modules, which are described in detail in Chapter 4.

- User Authentication (register, login and logout)
- Home Dashboard
- Tourist Places Browsing and Search
- Place Details
- Favorites Management
- Admin Authentication
- Tourist Place Management (add, edit and delete)
- Offline PWA Support

SOFTWARE REQUIREMENTS SPECIFICATION

2.1. User Requirements

The Tourist Guide platform recognises two categories of users: the User, who browses tourist places and manages favourites, and the Admin, who manages the tourist place information. The requirements of each category are stated below.

User Requirements

The user should be able to:

- Register an account on the platform.
- Log in to the platform using the registered credentials.
- Browse the tourist places available in the application.
- Search for tourist places.
- Filter the tourist places by category.
- View the details of a tourist place.
- Add a tourist place to the favourites list and remove it from the list.
- View the list of favourite places.
- Log out of the application.

Administrator Requirements

The administrator should be able to:

- Log in through a separate administrator login.
- View the Admin Dashboard.
- Add new tourist places.
- Edit existing tourist places.
- Delete tourist places.
- View the registered users and the system statistics.
- Log out of the administrative session.

The functions available to each actor, according to whether the actor is logged out, logged in as a user, or logged in as the administrator, are summarised in Table 2. A function marked Yes is available in that state; a function marked No is refused.

Function	Visitor (not logged in)	Logged-in User	Logged-in Admin
Register an account	Yes	No	No
User login	Yes	No	No
Browse, search and filter tourist places	No	Yes	No
View place details	No	Yes	No
Add, remove and view favorites	No	Yes	No
User logout	No	Yes	No
Admin login	Yes	No	No
View the Admin Dashboard	No	No	Yes
Add, edit and delete tourist places	No	No	Yes
View registered users and statistics	No	No	Yes
Admin logout	No	No	Yes

Table 2: Access Rights of Visitors, Users and Administrators

2.2. System Requirements

The system requirements describe the hardware and software environment required for the development, deployment and use of the proposed application.

Hardware Requirements

The minimum hardware configuration required for development and testing is given in Table 3.

Component	Minimum Specification
Processor	Dual-core processor, 2.0 GHz or higher
RAM	4 GB (8 GB recommended for development)
Storage	Minimum 10 GB of free space for development tools, project files and the database
Display	Standard monitor or laptop display, minimum resolution 1366 × 768
Input devices	Keyboard and pointing device
Network	Internet connection for installing dependencies and for live database updates
Test device	Android or iOS smartphone or tablet for verifying the responsive layout and installation

Table 3: Minimum Hardware Requirements

Software Requirements

The software environment required by the proposed system is given in Table 4.

Category	Requirement
Operating System	Windows 10 or later, Linux, or macOS
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox or Safari, in a version that supports service workers and the Cache Storage API
Code Editor	Visual Studio Code or an equivalent editor
Frontend Technologies	HTML5, CSS3 and JavaScript
Backend	Python 3 with the Flask framework
Python Dependencies	PyMySQL and Werkzeug
Database	MySQL as the primary database; SQLite as the automatic fallback
PWA Technologies	Web App Manifest, Service Worker, Cache Storage API
Hosting	A server capable of running a Flask application and serving it over HTTPS, which is required for service workers outside a local development machine

Table 4: Software Requirements

2.3. Design and Implementation Constraints

The following constraints are taken into account during the development of the proposed system.

- The front end is developed using HTML5, CSS3 and JavaScript.
- Python Flask is used for backend routing and application logic.
- MySQL is supported as the primary database, with an automatic fallback to SQLite when MySQL is not available.
- The interface must be responsive and compatible with desktop, tablet and mobile devices.
- The PWA features are limited to the Web App Manifest, the Service Worker and the Cache Storage API.
- Tourist destination information is managed only through the Admin Module.
- The application requires an internet connection for live database updates; cached resources can be used when the device is offline.
- Service workers are available only in a secure context, so the application must be served over HTTPS when it is deployed (the local development address is treated as secure by browsers).
- Passwords must not be stored in plain form, and administrative functions must not be reachable from an ordinary user session.
- The implementation is subject to the development time and the resources available within the semester.

2.4. Specific Requirements

This section states the interface, functional and non-functional requirements of the proposed system in detail.

2.4.1. Interface Requirements

The application should provide a clear and consistent interface for both categories of users. The major interface requirements are described below.

- **Registration Page:** Allows a new user to create an account by entering the required details, with validation of each field.
- **Login Page:** Allows a registered user to authenticate with the registered credentials.
- **Home Dashboard:** The starting point after login, from which the places list, the favourites and the logout option can be reached.
- **Tourist Places Page:** Presents the available tourist places as a list, with a search field and a category filter.
- **Place Details Page:** Displays the location, description, visiting timings, entry fee and category of a place, together with the control to add it to or remove it from the favourites.
- **Favorites Page:** Lists the places that the user has saved as favourites.
- **Admin Login Page:** Allows the administrator to authenticate through a separate login.
- **Admin Dashboard:** Presents the registered users, the system statistics and the controls to add, edit and delete tourist places.
- **Add / Edit Place Form:** Allows the administrator to enter or modify the details of a tourist place.
- **Error and Feedback Messages:** Present clear, non-technical messages for failed operations such as an incorrect login or an invalid form.

Navigation and Layout

All pages share a common layout with a header that carries the name of the application and the navigation links that are relevant to the role of the person who is logged in. A logged-in user sees links to the Home Dashboard, the tourist places, the favourites and the logout option, while the administrator sees links to the Admin Dashboard and the admin logout. Pages that require a login are not displayed to a visitor who is not logged in; the visitor is directed to the login page instead.

The tourist places are presented as a list of cards, each showing the principal details of a place and a link to its details page. The search field and the category filter are placed above the list so that the result of a change is visible immediately below the control that caused it.

The interface is developed using HTML5, CSS3 and JavaScript and follows a responsive design so that it presents an appropriate layout on desktop, tablet and mobile screens.

2.4.2. Functional Requirements

The functional requirements describe the operations that the system should perform.

- **FR1 – User Registration:** The system should allow a new user to register an account and should reject an email address that is already registered.
- **FR2 – User Login:** The system should allow a registered user to log in with the correct credentials and should refuse access when they are incorrect.
- **FR3 – User Logout:** The system should allow a user to log out and should end the user session.
- **FR4 – Home Dashboard:** The system should display the Home Dashboard to a logged-in user.
- **FR5 – Browse Tourist Places:** The system should display the list of available tourist places.
- **FR6 – Search Places:** The system should allow a user to search the tourist places and should display the matching places.
- **FR7 – Filter by Category:** The system should allow a user to filter the tourist places by category.
- **FR8 – View Place Details:** The system should display the location, description, visiting timings, entry fee and category of a selected place.
- **FR9 – Add to Favorites:** The system should allow a user to add a tourist place to the favourites list.
- **FR10 – Remove from Favorites:** The system should allow a user to remove a place from the favourites list.
- **FR11 – View Favorites:** The system should display the places that the user has saved as favourites.
- **FR12 – Admin Login:** The system should allow the administrator to log in through a separate administrator login and should refuse incorrect credentials.
- **FR13 – Admin Dashboard:** The system should display the Admin Dashboard to a logged-in administrator.
- **FR14 – Add Tourist Place:** The system should allow the administrator to add a new tourist place.
- **FR15 – Edit Tourist Place:** The system should allow the administrator to edit the details of an existing tourist place.
- **FR16 – Delete Tourist Place:** The system should allow the administrator to delete a tourist place.

- **FR17 – View Users and Statistics:** The system should allow the administrator to view the registered users and the system statistics.
- **FR18 – Admin Logout:** The system should allow the administrator to log out and should end the administrative session.
- **FR19 – Offline Access:** The system should keep cached pages and static resources accessible when the device is offline.
- **FR20 – Installation:** The system should allow the application to be installed on a device as a Progressive Web Application.

2.4.3. Non-Functional Requirements

Usability: The application should present a simple and uncluttered interface. A new user should be able to register, log in and find a tourist place without instruction, and the common actions of searching, filtering and saving a favourite should be reachable directly from the pages on which they are used.

Performance: Pages should load quickly on a normal connection, and cached static resources should be served from the browser without a network round trip. Searching and filtering should return results without noticeable delay for the volume of destination data held by the system.

Reliability: The system should carry out its intended functions consistently. A failed operation should leave the stored data unchanged, and the removal of a place should not leave dangling favourite records.

Availability: Cached pages and static resources should remain accessible when the device is offline. When the primary MySQL database cannot be reached, the application should continue to run using the SQLite fallback.

Security: Passwords should never be stored in plain form. User and administrator sessions should be separate, pages that require a login should be refused to an anonymous visitor, and administrative functions should be reachable only from an administrative session. Database queries should use parameter binding so that user input cannot alter the meaning of a query.

Privacy: A user's favourites list should be visible only to that user, and the stored credentials of users and administrators should not be displayed in any screen.

Maintainability: The application should be organised into clearly separated modules, so that an individual feature can be modified without affecting unrelated parts of the system. Access to the database should be confined to a single connection component so that the choice between MySQL and SQLite does not affect the rest of the code.

Scalability: The design should permit a growing number of tourist places and registered users. Because the primary database is MySQL, the data store can be moved to a dedicated database server as the volume of data grows.

Compatibility: The application should function correctly on current versions of the major browsers on both desktop and mobile platforms. Where a browser does not support a PWA capability such as installation, the rest of the application should continue to work.

Responsiveness: The interface should adapt to desktop, tablet and mobile screen widths, presenting a single-column layout on small screens and a multi-column layout on larger screens.

Portability: As a web application the system should run on any device with a supported browser and should not require a platform-specific installation package on the client.

SYSTEM DESIGN AND ANALYSIS

3.1. System Architecture Design

The Tourist Guide application follows a layered architecture in which each layer has a defined responsibility and communicates only with the layers adjacent to it. This separation keeps the presentation logic independent of the storage mechanism and confines all database access to a single point, which simplifies both testing and future modification. The architecture of the system is shown in Figure 1.

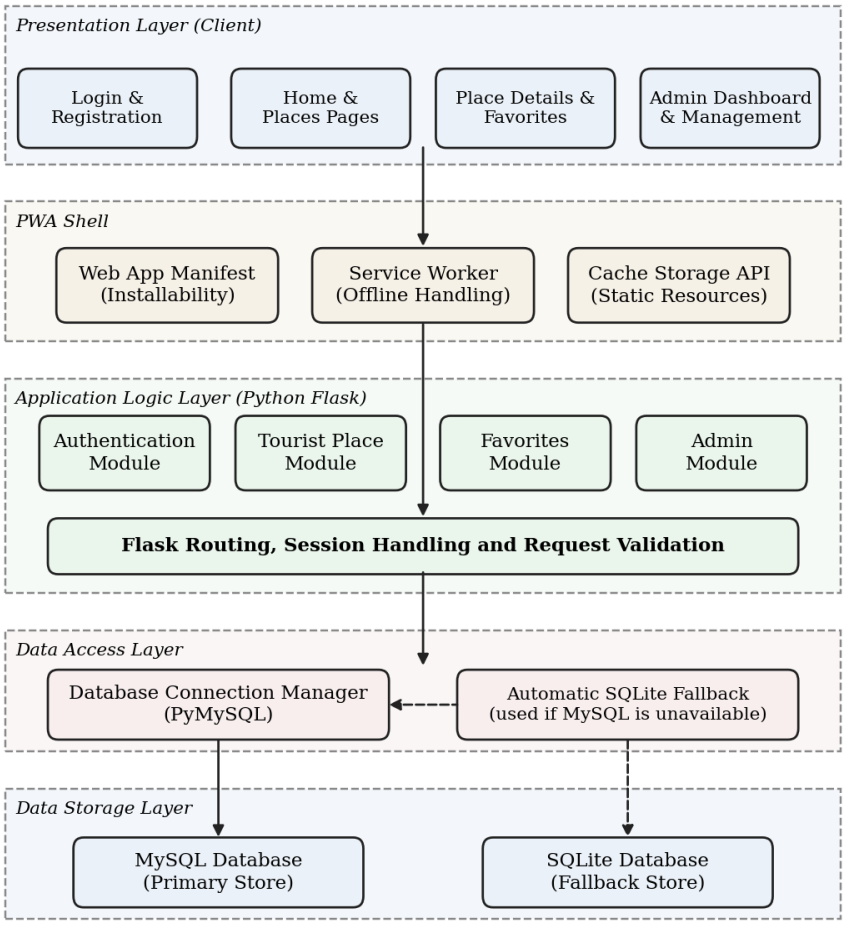


Figure 1: System Architecture Diagram

The five layers of the architecture are described below.

Presentation Layer

The presentation layer contains everything that the user sees and interacts with. It consists of the registration and login pages, the Home Dashboard and the places pages, the place details and favourites pages, and the administrative pages. This layer is built from HTML5, CSS3 and JavaScript. It performs no storage of its own; it displays the information supplied by the application layer and forwards user actions to it.

Progressive Web Application Shell

The shell layer contains the components that give the application its progressive characteristics. The Web App Manifest supplies the metadata required for installation. The Service Worker intercepts requests made by the pages and serves cached responses when the network is unavailable. The Cache Storage API holds the cached pages and static resources. This layer sits between the interface and the server so that the pages can be opened whether or not the device is currently online.

Application Logic Layer

The application logic layer is implemented in Python with Flask and contains the working parts of the system. Flask routing receives each request and directs it to the appropriate module, and Flask sessions record whether a user or an administrator is logged in. The authentication module handles registration, login and logout. The tourist place module handles browsing, searching, filtering and the place details. The favourites module adds, removes and lists the favourite places of a user. The admin module handles the administrator login, the Admin Dashboard, the management of tourist places and the display of users and statistics.

Data Access Layer

The data access layer is the only part of the system that communicates with the database. A database connection manager first attempts to connect to MySQL using the PyMySQL driver. If MySQL cannot be reached, the manager switches automatically to SQLite, so that the modules above it are written once and operate unchanged on either database. All queries are issued with bound parameters.

Data Storage Layer

The data storage layer holds all persistent information. MySQL is the primary store for the users, administrators, tourist places and favourites. SQLite acts as the fallback store with the same logical structure, which is described in Section 3.3.

3.2. Workflow Diagrams

This section presents the workflows that describe how the system is used, followed by a use case model that summarises the interaction between the actors and the system.

User Workflow

Figure 2 shows the sequence of activities carried out by a user during a typical session. The user opens the application, and a user who is not yet registered creates an account and then logs in. After login the Home Dashboard is displayed. The user searches or filters the tourist places, opens a place to view its details, and may add the place to or remove it from the favourites. The user can then view the favourites list, continue to use cached pages when offline, and finally log out.

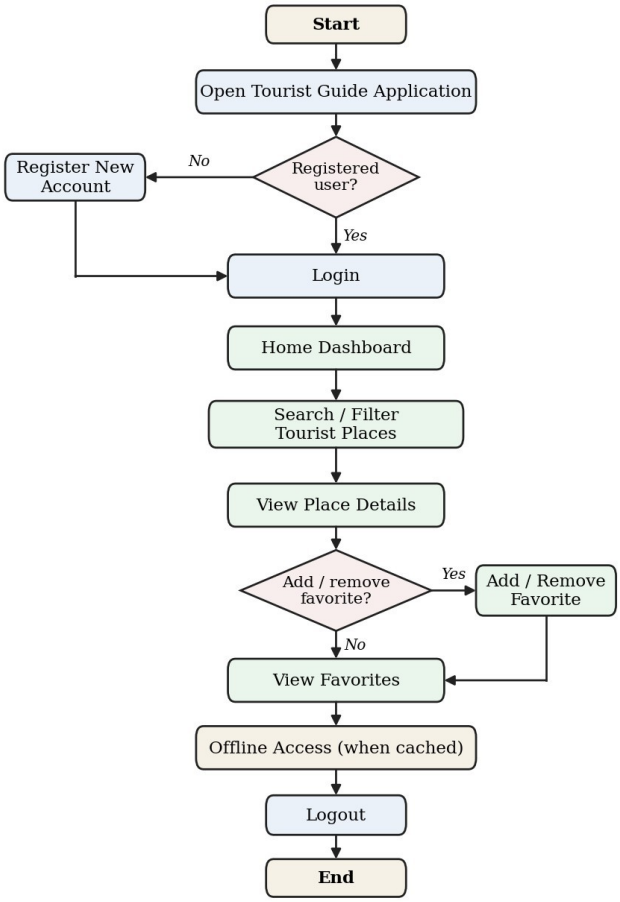


Figure 2: User Workflow Diagram

Admin Workflow

Figure 3 shows the workflow of the administrator. The administrator logs in through the separate admin login; if the credentials are not valid, access is refused and the login is shown again. After a successful login the Admin Dashboard is displayed, from which the administrator can add, edit or delete a tourist place or view the registered users and system statistics. The session ends when the administrator logs out.

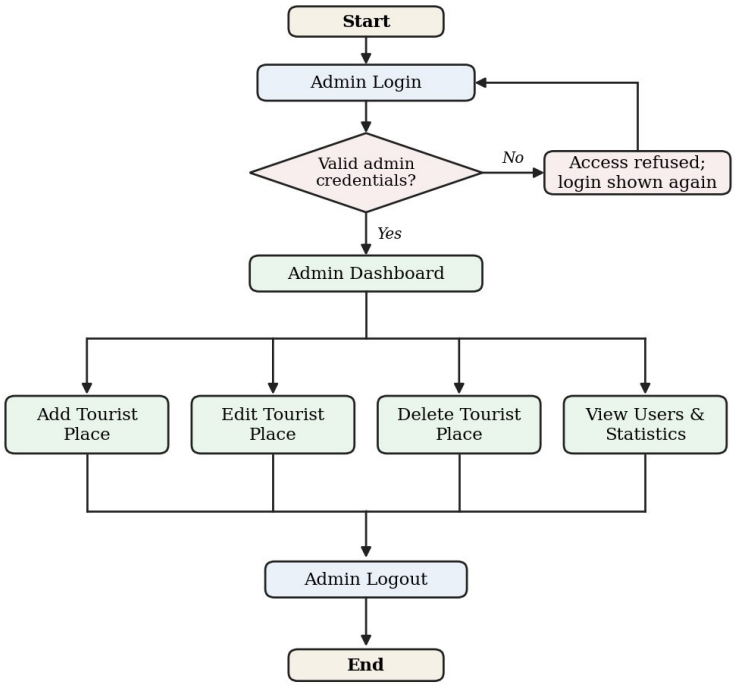


Figure 3: Admin Workflow Diagram

Offline Request Handling Workflow

Figure 4 describes how a request made by the browser is handled by the Service Worker. The Service Worker intercepts the request and checks whether the requested resource is available in Cache Storage. If it is, the cached response is returned. If it is not, the Service Worker tries the network; when the network is available the resource is fetched from the Flask server, static resources are stored in the cache for later use, and the response is returned. When the resource is not cached and the network is unavailable, the request cannot be completed, because live database content requires an internet connection.

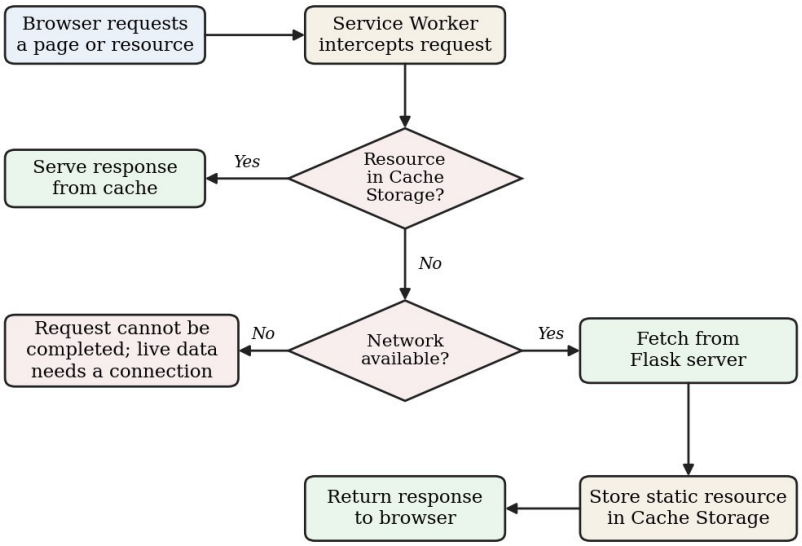


Figure 4: Offline Request Handling Workflow

Use Case Model

Figure 5 shows the use case model of the system. The User is the primary actor, associated with the use cases for registering, logging in, browsing, searching and filtering tourist places, viewing place details, managing and viewing favourites, using cached pages offline and logging out. The Administrator is a second actor associated with the administrative use cases: admin login, viewing the dashboard, adding, editing and deleting tourist places, viewing users and statistics, and admin logout. The two sets are disjoint, reflecting the requirement that administrative functions must not be reachable from an ordinary user session.

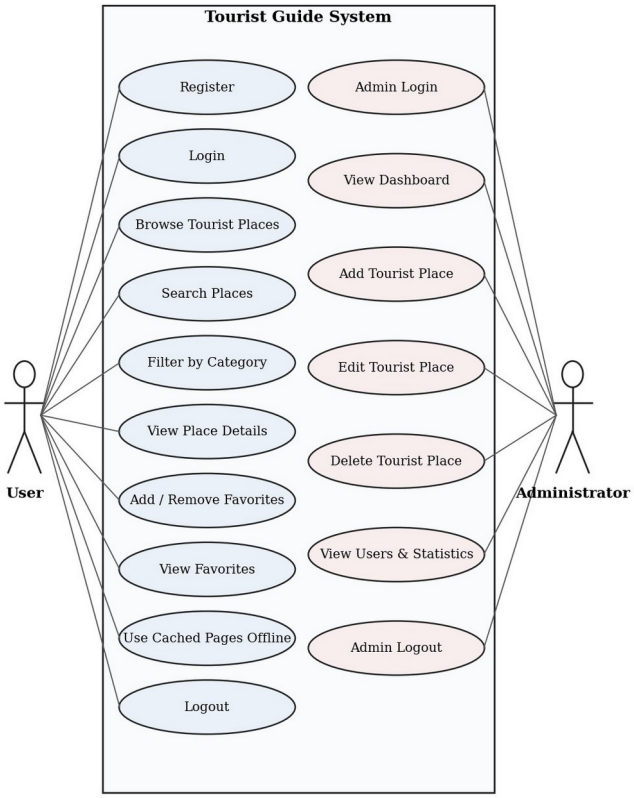


Figure 5: Use Case Diagram

3.3. Database Design

The system stores its data in a relational database. MySQL is the primary database and SQLite is the automatic fallback, and the same logical schema is created in either engine. The schema consists of four tables: users, tourist_places, favorites and admins. The entity-relationship diagram is shown in Figure 6.

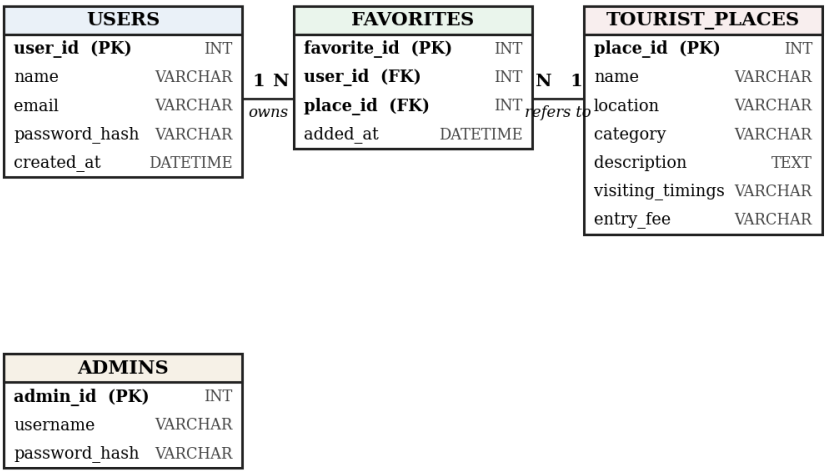


Figure 6: Entity-Relationship Diagram

The **users** table holds the registered users, the **tourist_places** table holds the destination information maintained by the administrator, the **favorites** table records which user has saved which place, and the **admins** table holds the administrator accounts. The favorites table resolves the many-to-many relationship between users and tourist places: one user can save many places, and one place can be saved by many users. The admins table is independent of the other tables, because administrators are not users of the favourites feature.

Data Dictionary

The structure of each table is given in Tables 5 to 8. Column types are given in MySQL notation; in SQLite the equivalent types are used.

Field Name	Data Type	Constraints	Description
user_id	INT	Primary key, auto-increment	Unique identifier of the user
name	VARCHAR(100)	Not null	Name of the registered user
email	VARCHAR(120)	Not null, unique	Email address used to log in
password_hash	VARCHAR(255)	Not null	Hashed password; the plain password is never stored
created_at	DATETIME	Default current time	Date and time of registration, shown in the user list

Table 5: Structure of the users Table

Field Name	Data Type	Constraints	Description
place_id	INT	Primary key, auto-increment	Unique identifier of the tourist place
name	VARCHAR(150)	Not null	Name of the tourist place
location	VARCHAR(150)	Not null	Location of the place
category	VARCHAR(50)	Not null	Category used by the category filter
description	TEXT	Not null	Description of the place
visiting_timings	VARCHAR(100)	Not null	Visiting timings of the place
entry_fee	VARCHAR(50)	Not null	Entry fee, stored as text so that a free entry can also be recorded

Table 6: Structure of the tourist_places Table

Field Name	Data Type	Constraints	Description
favorite_id	INT	Primary key, auto-increment	Unique identifier of the favourite record
user_id	INT	Foreign key to users	User who saved the place
place_id	INT	Foreign key to tourist_places	Place that was saved
added_at	DATETIME	Default current time	Date and time at which the place was saved

Table 7: Structure of the favorites Table

Field Name	Data Type	Constraints	Description
admin_id	INT	Primary key, auto-increment	Unique identifier of the administrator
username	VARCHAR(50)	Not null, unique	Username used for the administrator login
password_hash	VARCHAR(255)	Not null	Hashed administrator password

Table 8: Structure of the admins Table

Integrity Rules

- The email column of the users table is unique, so a second account cannot be registered with an address that is already in use.
- The pair of columns user_id and place_id in the favorites table is unique, so a place cannot be saved twice by the same user.
- The foreign keys of the favorites table are defined so that deleting a tourist place or a user also removes the related favourite records.
- Passwords of users and administrators are stored only as hashes generated with the Werkzeug security helpers.
- Administrator accounts are provisioned directly in the admins table; there is no public administrator registration.

3.4. Data Flow Diagrams

Data flow diagrams describe how data moves between the external entities, the processes of the system and its data stores. Figure 7 is the context diagram (level 0). It shows the system as a single process that exchanges data with the two external entities: the User sends registration, login, search and favourite requests and receives place lists, details and favourites, while the Administrator sends credentials and place data and receives the dashboard, the user list and the statistics.



Figure 7: Data Flow Diagram – Level 0 (Context Diagram)

Figure 8 expands the user side of the system into level 1. Process 1.0 authenticates users against the users store, process 2.0 reads the tourist places store to answer browse, search and detail requests, and process 3.0 reads and writes the favorites store, using the tourist places store to present the saved places.

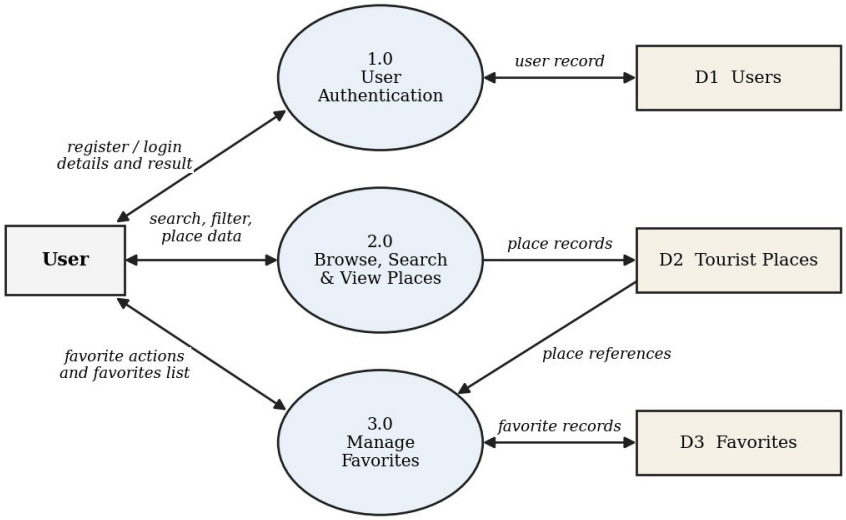


Figure 8: Data Flow Diagram – Level 1 (User Processes)

Figure 9 expands the administrator side. Process 4.0 authenticates the administrator against the admins store, process 5.0 reads and writes the tourist places store when places are added, edited or deleted, and process 6.0 reads the users, tourist places and favorites stores to produce the user list and the system statistics.

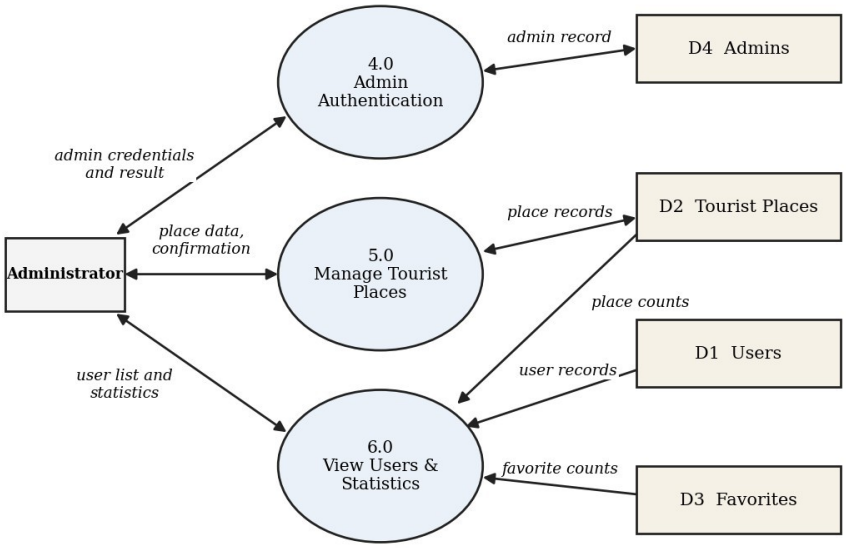


Figure 9: Data Flow Diagram – Level 1 (Admin Processes)

3.5. Sequence Diagrams

Sequence diagrams show the order of the messages exchanged between the user, the browser, the Flask server and the database for the principal operations. Figure 10 shows the login sequence. The user submits the credentials from the browser, the authentication module queries the user record, verifies the submitted password against the stored hash, creates the session on success and redirects the browser to the Home Dashboard.

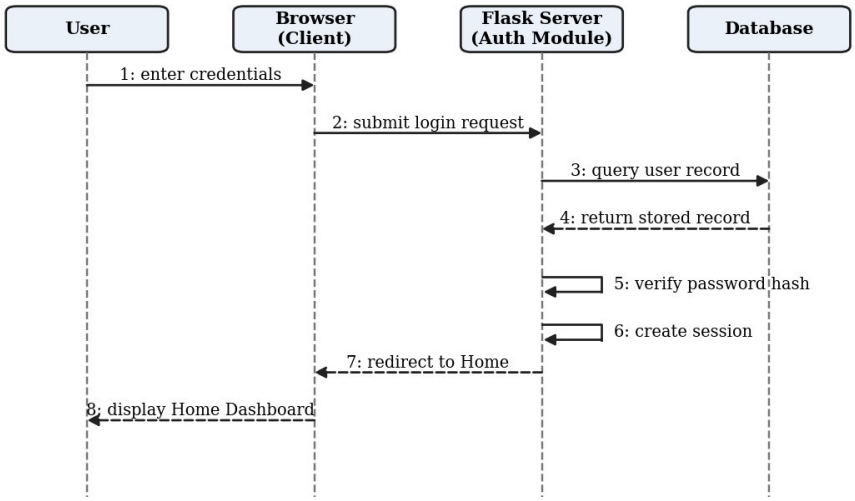


Figure 10: Sequence Diagram – User Login

Figure 11 shows the sequence for browsing and saving a place. A search or category selection is sent to the server, which queries the tourist places and returns the matching results. When the user opens a place, the server fetches that place record and returns the details page. When the user chooses to add the place to the favourites, the server inserts a favourite record and confirms the action.

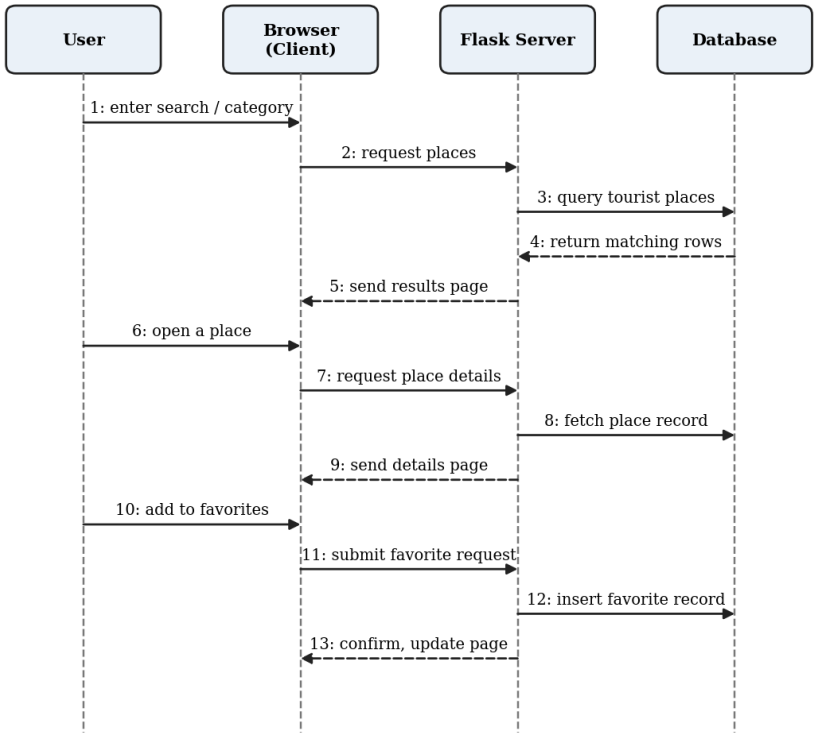


Figure 11: Sequence Diagram – Search, View Details and Add to Favorites

Figure 12 shows the sequence for adding a tourist place. The administrator submits the place details, the admin module first verifies that the request belongs to an administrative session, the place record is inserted in the database, and the updated place list is returned to the browser.

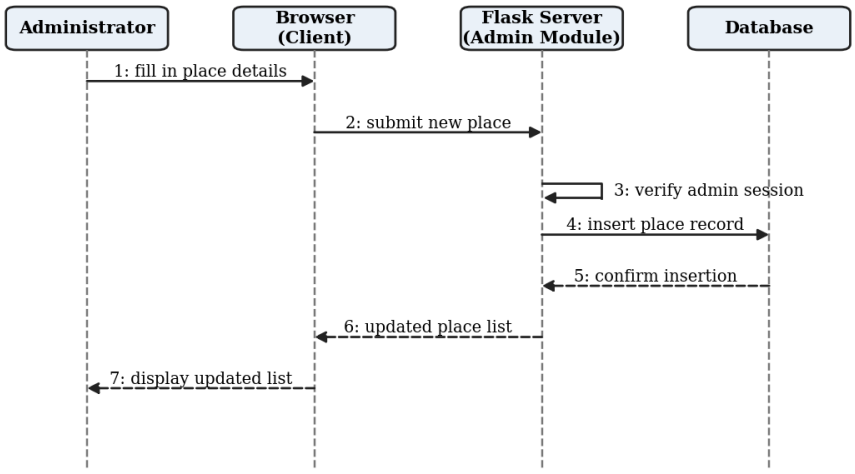


Figure 12: Sequence Diagram – Admin Adds a Tourist Place

3.6. Security and Session Design

Security is addressed in the design of the application and is not left to the individual pages. The principal design decisions are described below.

Password Protection

Passwords of users and administrators are never stored as typed. When an account is created, the password is converted to a hash using the security helpers of Werkzeug, and only the hash is written to the database. At login the submitted password is checked against the stored hash, so that the original password cannot be read from the database even by someone who has access to it.

Session Management and Role Separation

After a successful login the Flask session records the identity of the person and whether the person is a user or the administrator. The user session and the administrative session are kept separate: a user session cannot open an administrative page, and an administrative session is not treated as a user session. Every protected route checks the session before it performs any action, and the session is cleared when the person logs out.

Protection of Stored Data

All queries are written with bound parameters, so that text entered into a form or a search field is treated as data and cannot change the meaning of a query. The favourites list of a user is always read and changed using the identity held in that user’s own session, so one user cannot view or alter the favourites of another.

Secure Transport

When the application is deployed it is served over HTTPS. This protects credentials in transit and is also required by browsers before a Service Worker can be registered.

IMPLEMENTATION

4.1. Development Approach

The system is developed using an incremental and iterative approach. The complete set of requirements is divided into a sequence of increments, each of which produces a working version of the application with a further group of features in place. Every increment passes through its own cycle of analysis, design, coding and testing, and the version produced at the end of an increment is reviewed before the next increment begins.

This approach is appropriate for the present project for three reasons. First, the requirements are well understood in outline but may be refined as the application takes shape, and an incremental approach allows such refinements to be absorbed without redesigning the whole system. Second, a working version exists from an early stage, which makes it possible to find problems before a large amount of effort has been committed. Third, the project is bounded by the duration of a semester, and dividing the work into increments keeps progress measurable against that limit.

Increment 1: Project Setup, Database and Authentication

The first increment establishes the foundation of the application. The Flask project structure is created, the database connection manager is written with the MySQL connection and the automatic SQLite fallback, and the tables of the schema are created. The registration, login and logout of users are then implemented, with passwords stored as hashes and with session handling.

Increment 2: Home Dashboard and Tourist Places

The second increment adds the central browsing features. The Home Dashboard, the tourist places list, the search facility, the category filter and the place details page are implemented, together with the responsive page layout and stylesheet that all later pages reuse.

Increment 3: Favorites Management

The third increment adds the favourites feature. A logged-in user can add a place to the favourites, remove it, and view the list of saved places, and the integrity rules on the favorites table are put in place.

Increment 4: Admin Module

The fourth increment implements the administrative side of the system. It adds the administrator login with a session that is separate from the user session, the Admin Dashboard, the forms for adding, editing and deleting tourist places, and the display of the registered users and the system statistics.

Increment 5: Progressive Web Application Features

The fifth increment converts the application into an installable Progressive Web Application. The Web App Manifest and the icon set are added, the Service Worker is written and registered, the static resources and visited pages are cached with the Cache Storage API, and the caching strategy is verified by testing the application with the network disabled.

Increment 6: Integration and Testing

The final increment integrates the modules and tests the application as a whole against the test cases of Chapter 5, including the responsive layout at different screen widths, the fallback from MySQL to SQLite, and the separation between user and administrator sessions.

Proposed Project Organisation

The application is organised as a Flask project in which server-side code, page templates and static files are kept apart.

- **Application code:** the Flask application file creates the application, registers the routes of the modules and holds the session configuration; the database connection manager is kept in its own file.
- **Templates:** the HTML5 pages of the user side and of the administrative side, built on a common base layout.
- **Static files:** the CSS3 stylesheets, the JavaScript files and the application icons.
- **PWA files:** the Web App Manifest and the Service Worker script, which are served from the root of the application so that the Service Worker can control every page.

Practices Followed Throughout Development

- **Modular organisation:** Each functional area is implemented as a separate module with a defined interface, and modules communicate only through those interfaces.
- **Parameterised queries:** Every database query is written with bound parameters so that user input is never combined directly into a query.
- **Separation of roles:** User routes and administrator routes check their own session flag, so that an administrative page cannot be opened from a user session.
- **Mobile-first styling:** Styles are written first for narrow screens and extended for larger screens through media queries.
- **Continuous testing:** Each module is tested as it is written, and the full application is re-tested at the end of every increment.
- **Documentation:** Requirements, design decisions and interfaces are recorded as the work proceeds rather than at the end of the project.

4.2. Module Description

The system is divided into eight modules. Each module is responsible for a single area of functionality and exposes a defined interface to the other modules. All modules reach the database only through the shared database connection manager.

M1: User Authentication Module

This module is responsible for the identity of ordinary users. It registers a new user after validating the submitted details and rejecting an email address that is already registered, and it stores the password only as a hash. It authenticates a returning user by comparing the submitted password with the stored hash, establishes a session on success and reports a failure without revealing which of the two credentials was wrong. It ends the session on logout and refuses access to protected pages when no user is logged in.

M2: Home Dashboard Module

This module prepares the Home Dashboard that is displayed after a successful login. The dashboard is the starting point of the application and provides access to the tourist places, the favourites and the logout option, so that the main activities are reachable directly from one page.

M3: Tourist Places Browsing and Search Module

This module retrieves the tourist places from the database and presents them as a list. It applies the search text entered by the user and the selected category to the query and returns only the matching places. When no place matches, the module returns an empty result so that the interface can inform the user.

M4: Place Details Module

This module retrieves a single tourist place and presents its location, description, visiting timings, entry fee and category on the details page. It also determines whether the place is already in the favourites of the logged-in user, so that the page can offer the correct control for adding or removing it.

M5: Favorites Management Module

This module maintains the favourites of a user. It adds a place to the favorites table, removes it again, and retrieves the saved places of the logged-in user for the favourites page. It prevents the same place from being saved twice by the same user, and it only ever reads or changes the favourites of the user who owns the current session.

M6: Admin Authentication and Dashboard Module

This module provides the entry to the administrative side of the system. It authenticates the administrator against the admins table, establishes an administrative session that is separate from the user session, and ends it on admin logout. It also prepares the Admin Dashboard,

which lists the registered users and presents system statistics such as the number of users, tourist places and saved favourites.

M7: Tourist Place Management Module

This module lets the administrator maintain the destination information. It validates and stores a new tourist place, loads an existing place into the edit form and saves the changes, and deletes a place together with its related favourite records. All of its functions require an administrative session.

M8: Offline PWA Support Module

This module contains the Web App Manifest, the Service Worker and the supporting script that registers it. It caches the static resources and the pages that have been visited, serves cached responses when the network is unavailable, replaces the cache when a new version of the application is deployed, and allows the browser to offer the application for installation.

A summary of the modules and their principal responsibilities is given in Table 9.

No.	Module	Principal Responsibility
M1	User Authentication	Registration, login and logout of users; password hashing and session handling
M2	Home Dashboard	Starting page after login and access to the main features
M3	Tourist Places Browsing and Search	Listing of places, search and category filter
M4	Place Details	Display of the location, description, timings, entry fee and category of a place
M5	Favorites Management	Adding, removing and listing the favourite places of a user
M6	Admin Authentication and Dashboard	Administrator login and logout, separate admin session, user list and statistics
M7	Tourist Place Management	Adding, editing and deleting tourist places
M8	Offline PWA Support	Manifest, Service Worker, caching of resources and installation

Table 9: Summary of System Modules

The functional requirements of Section 2.4.2 are realised by the modules as shown in Table 10, together with the database tables that each module reads or writes. Every functional requirement is assigned to exactly one module, which allows each requirement to be traced to the code that implements it and to the test cases that verify it.

Module	Functional Requirements	Tables Accessed
M1: User Authentication	FR1, FR2, FR3	users
M2: Home Dashboard	FR4	None
M3: Tourist Places Browsing and Search	FR5, FR6, FR7	tourist_places
M4: Place Details	FR8	tourist_places, favorites
M5: Favorites Management	FR9, FR10, FR11	favorites, tourist_places
M6: Admin Authentication and Dashboard	FR12, FR13, FR17, FR18	admins, users, tourist_places, favorites
M7: Tourist Place Management	FR14, FR15, FR16	tourist_places, favorites
M8: Offline PWA Support	FR19, FR20	None (browser Cache Storage)

Table 10: Traceability of Functional Requirements to Modules

Interaction Between Modules

The modules cooperate through the Flask session and through the database. When M1 or M6 authenticates a person, it records in the session whether the person is a user or the administrator, and every other module checks that record before it acts. M3 and M4 supply the places that M5 lists as favourites, and M7 changes the places that M3 and M4 later present, so a place added by the administrator appears in the search results of users without any further step. When M7 deletes a place, the favourite records that refer to it are removed with it, and M6 reads the same tables to report the statistics.

Shared Component: Database Connection Manager

The modules above do not open database connections themselves. They obtain a connection from a shared database connection manager, which reads the connection settings of the MySQL server and attempts to connect to it through PyMySQL. If the connection cannot be established, the manager opens a SQLite database file instead and creates the tables of the schema if they do not yet exist. The manager also hides the small differences between the two engines, such as the style of the parameter placeholders, so that the queries written in the modules behave in the same way on either database. Because this behaviour is confined to one component, the automatic fallback required by the design constraints does not affect the logic of any other module.

4.3. User Interface Screenshots

To illustrate the outcome of the implementation described in the preceding sections, this section presents a set of screenshots taken from the working prototype of the Tourist Guide application. The screenshots below follow the principal screens in the order in which a user typically encounters them, beginning with the user side of the application and ending with the Admin Dashboard.

Figure 13 shows the registration and login page. A new user creates an account by entering the required details, and a registered user logs in with the registered credentials. An incorrect login produces a clear, non-technical message.

[Paste screenshot here: Registration and login page]

Figure 13: Registration and Login Page of the Tourist Guide Application

Figure 14 shows the Home Dashboard presented after a successful login. It is the starting point of the application and gives access to the tourist places, the favourites and the logout option.

[Paste screenshot here: Home Dashboard]

Figure 14: Home Dashboard

Figure 15 shows the tourist places page. The available places are listed as cards, with a search field and a category filter placed above the list.

[Paste screenshot here: Tourist Places page with search and category filter]

Figure 15: Tourist Places Page with Search and Category Filter

Figure 16 shows the details page of a tourist place, which displays its location, description, visiting timings, entry fee and category, together with the control for adding the place to or removing it from the favourites.

[Paste screenshot here: Place Details page]

Figure 16: Place Details Page

Figure 17 shows the favourites page, which lists only the places that the logged-in user has saved.

[Paste screenshot here: Favorites page]

Figure 17: Favorites Page

Figure 18 shows the Admin Dashboard, which presents the registered users, the system statistics and the controls for adding, editing and deleting tourist places.

[Paste screenshot here: Admin Dashboard]

Figure 18: Admin Dashboard

TESTING

5.1. Testing Objectives

Testing is carried out in order to establish that the application behaves as the requirements state, to reveal defects before the system is submitted, and to confirm that the system continues to behave correctly after each increment is added. The objectives of the testing activity are stated below.

- To verify that every functional requirement stated in Section 2.4.2 is implemented and behaves as specified.
- To confirm that invalid or incomplete input is rejected with a clear message and does not leave the system in an inconsistent state.
- To verify that authentication and session management restrict access to the appropriate role, and that administrative pages cannot be reached from an ordinary user session.
- To verify that browsing, searching and filtering return the correct tourist places.
- To verify that favourites are added, removed and listed correctly for the logged-in user.
- To verify that the administrator can add, edit and delete tourist places and that the changes are visible to users.
- To confirm that the application falls back to SQLite when MySQL is unavailable.
- To verify that cached pages and static resources remain accessible while the device is offline and that the application can be installed.
- To confirm that the interface presents an appropriate layout on desktop, tablet and mobile screen widths.
- To establish, through regression testing, that features implemented in earlier increments continue to work after later increments are added.

Testing is conducted at four levels. Unit testing examines each module in isolation as it is written. Integration testing examines the interaction between modules, particularly between the Flask modules and the database connection manager. System testing examines the application as a whole against the requirements specification. User acceptance testing places the application before a small group of users to confirm that it is usable for its intended purpose.

Test Environment

The tests are planned for the following environment.

- A desktop browser with developer tools, used to inspect Cache Storage and the Service Worker and to simulate the offline condition.
- A smartphone or tablet browser, used to check the responsive layout and the installation of the application.

- A running MySQL server for normal operation, which is stopped for the test of the SQLite fallback.
- A set of sample tourist places covering more than one category, one registered test user and one administrator account.

Test Data

The sample tourist places are chosen so that every test can be checked against a known answer. At least two categories are present so that the category filter can be shown to include and to exclude places, and at least one place is added to the favourites of one test user but not of another, so that the privacy of the favourites list can be confirmed.

5.2. Hypothetical Test Cases (Input, Expected Output, Success-Failure Criteria)

The test cases in Table 11 have been prepared in advance of the final integration. Each case states the input to be supplied, the output that the system is expected to produce, and the criterion by which the case is judged to have passed or failed.

ID	Test Case	Input	Expected Output	Success / Failure Criteria
TC01	Registration with valid details	Name, unused email address and a password	Account created and the user can log in	Pass if the account is created; fail if registration is rejected
TC02	Registration with a duplicate email	An email address that is already registered	Registration refused with a message that the address is in use	Pass if no second account is created; fail otherwise
TC03	User login with correct credentials	Registered email and correct password	Session established and Home Dashboard displayed	Pass if access is granted; fail if valid credentials are refused
TC04	User login with an incorrect password	Registered email and a wrong password	Access refused with a general failure message	Pass if access is refused and the message does not reveal which field was wrong
TC05	User logout	Logout selected	Session ended and login page displayed	Pass if a protected page cannot be reopened afterwards; fail if it can
TC06	Access to a protected page without login	Favourites page requested without logging in	User redirected to the login page	Pass if the page is not displayed; fail if it is displayed
TC07	Browse tourist places	Places page opened	List of available tourist places displayed	Pass if the stored places are listed; fail if the list is empty or incorrect
TC08	Search for a place	Search text matching at least one place	Only the matching places are listed	Pass if every matching place is returned; fail if a match is omitted
TC09	Search with no match	Search text matching no place	Empty result with an informative message	Pass if no unrelated place is shown and the page does not fail

ID	Test Case	Input	Expected Output	Success / Failure Criteria
TC10	Filter by category	A category selected in the filter	Only places of that category are listed	Pass if the filter is applied correctly; fail if unrelated places appear
TC11	View place details	A place selected from the list	Location, description, timings, entry fee and category displayed	Pass if all details of the place are shown; fail if any is missing
TC12	Add a place to favourites	Add to Favourites selected on a place	Place saved and shown in the favourites list	Pass if the place appears in the list; fail if it does not
TC13	Remove a place from favourites	Remove selected for a saved place	Place removed from the favourites list	Pass if the place disappears from the list; fail if it remains
TC14	View favourites	Favourites page opened	Only the places saved by the logged-in user are listed	Pass if the list is correct for that user; fail if another user's places appear
TC15	Admin login with valid credentials	Valid administrator username and password	Admin Dashboard displayed	Pass if access is granted; fail if valid credentials are refused
TC16	Admin login with invalid credentials	Valid username and a wrong password	Access refused and login shown again	Pass if access is refused; fail if the dashboard is displayed
TC17	Admin page requested from a user session	Admin Dashboard address requested while logged in as a user	Access refused	Pass if no administrative page is displayed; fail otherwise
TC18	Add a tourist place	Valid details of a new place entered by the admin	Place stored and visible in the places list	Pass if the new place is listed for users; fail if it is not
TC19	Edit a tourist place	A change to the details of an existing place	Modified details saved and displayed	Pass if the change persists after reload; fail if it is lost
TC20	Delete a tourist place	Delete selected for a place by the admin	Place removed from the list and from favourites	Pass if the place and its favourite records are gone; fail if either remains
TC21	View users and statistics	Admin Dashboard opened	Registered users and system statistics displayed	Pass if the figures match the stored data; fail otherwise
TC22	Offline access to cached pages	Previously visited page opened with the network disabled	Page displayed from Cache Storage	Pass if the cached page opens; fail if it cannot be displayed
TC23	Install the application	Installation accepted from the browser prompt	Application installed and opened in its own window	Pass if installation completes; fail if the prompt does not appear or installation fails
TC24	Database fallback	Application started while MySQL is unavailable	Application runs using SQLite	Pass if pages and data operations work; fail if the application stops
TC25	Responsive layout	Application opened on desktop, tablet and mobile widths	Layout adapts with all controls reachable	Pass if no horizontal scrolling or overlap occurs; fail otherwise

Table 11: Hypothetical Test Cases

Recording of Results

When the tests are executed, the outcome of each case is recorded as pass or fail together with a short remark. A failed case is traced to the module that is responsible for it using Table 10, the defect is corrected, and the case is executed again. Because corrections can affect earlier work, the cases of earlier increments are repeated at the end of each increment as part of regression testing.

CONCLUSION

The Tourist Guide project set out to address a difficulty familiar to almost every traveller: information about tourist destinations is spread across many websites, is presented in different forms, and cannot be reached when the network is poor. The system described in this report responds by placing browsing, searching, filtering, viewing details and saving favourites within a single application, and by making the application installable and partly usable offline.

The work documented here covers the analysis and design of the system. The requirements were recorded as user, system and interface requirements, twenty functional requirements and eleven categories of non-functional requirement. A five-layer architecture was designed in which the presentation layer, the progressive web application shell, the application logic layer, the data access layer and the data storage layer each hold a defined responsibility. The behaviour of the system was modelled through a user workflow, an admin workflow, an offline request handling workflow, a use case model, data flow diagrams at two levels and three sequence diagrams. The data was organised in a relational schema of four tables, and the implementation was divided into eight modules and planned as six increments. A test strategy was prepared consisting of four levels of testing and twenty-five hypothetical test cases with defined success and failure criteria.

Three design decisions deserve particular mention. Confining all database access to one connection manager, with an automatic fallback from MySQL to SQLite, means that the rest of the application is written once and runs in either environment. Separating the user session from the administrative session and protecting every administrative function behind it keeps the destination information under the control of the administrator. Using the Web App Manifest, the Service Worker and the Cache Storage API allows the application to be installed and its cached resources to remain available when the traveller has no connection.

The project has also provided practical experience of a complete software engineering life cycle: documenting requirements in a structured form, designing a layered architecture and a relational database, defining the modules of a web application, planning an incremental implementation and preparing a test strategy in advance. The design is complete and consistent, and it provides a sound foundation for the implementation and for the enhancements described in the next chapter.

FUTURE ENHANCEMENTS

The following enhancements were identified during the design of the system. They lie outside the scope of the present work but are consistent with the architecture described in Chapter 3 and could be incorporated in later versions.

GPS Geolocation and Nearby Places

Using the location of the device, the application could show the tourist places that lie near the traveller, so that a visitor in an unfamiliar town can immediately see what is worth visiting close by. The existing place records could be extended with coordinates, and the distance to each place could then be calculated and used to order the list.

Interactive Maps and Route Navigation

Embedding interactive maps would show each destination on a map, and route navigation would guide the traveller from the present position to the chosen place. This would remove the need to copy a location into a separate navigation application.

Push Notifications and Travel Reminders

Push notifications delivered through the Service Worker could remind a traveller about planned visits or about the places saved as favourites. Such notifications would require the traveller’s permission and a server-side component that sends the messages at the right time.

User Reviews and Ratings

Allowing users to rate destinations and to write reviews would add the experience of other travellers to the factual description maintained by the administrator. Reviews would need a new table related to both users and tourist places, together with a way for the administrator to remove unsuitable entries.

Multi-Language Support

Presenting the interface and the destination information in several languages would make the application useful to a wider range of travellers.

Travel Itinerary Planner

An itinerary planner would let a user arrange selected places into a day-by-day plan for a trip. The favourites list would be a natural starting point, since it already holds the places that the user has chosen.

Weather Information

Showing the weather at a destination would help a traveller choose the best time for a visit. The information could be obtained from an external weather service and displayed on the place details page.

AI-Based Tourist Destination Recommendations

Recommendations based on the places a user has browsed or saved could suggest further destinations that match the user's interests.

Hotel and Restaurant Information

Adding information about hotels and restaurants near each destination would extend the application from a place guide to a more complete travel companion. These records could be maintained by the administrator in the same way as the tourist places.

Advanced Offline Data Synchronization

The present design caches pages and static resources. A more advanced synchronisation mechanism could store data on the device and synchronise changes with the server when the connection returns, so that more of the application remains usable offline.

BIBLIOGRAPHY

The following sources were consulted during the preparation of this report.

[1] Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner's Approach, 9th Edition, McGraw-Hill Education, 2020.

[2] Sommerville, I., Software Engineering, 10th Edition, Pearson Education, 2016.

[3] Rajib Mall, Fundamentals of Software Engineering, 5th Edition, PHI Learning, 2018.

[4] IEEE Computer Society, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.

[5] Booch, G., Rumbaugh, J. and Jacobson, I., The Unified Modeling Language User Guide, 2nd Edition, Addison-Wesley, 2005.

[6] Pallets Projects, Flask Documentation, https://flask.palletsprojects.com/

[7] Pallets Projects, Werkzeug Documentation, https://werkzeug.palletsprojects.com/

[8] PyMySQL Documentation, https://pymysql.readthedocs.io/

[9] Oracle Corporation, MySQL Reference Manual, https://dev.mysql.com/doc/

[10] SQLite Documentation, https://www.sqlite.org/docs.html

[11] Mozilla Developer Network, Progressive Web Apps – Guides and Reference, https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

[12] Mozilla Developer Network, Service Worker API, https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

[13] Mozilla Developer Network, Cache API, https://developer.mozilla.org/en-US/docs/Web/API/Cache

[14] Mozilla Developer Network, Web App Manifest, https://developer.mozilla.org/en-US/docs/Web/Manifest

[15] Google Developers, Learn PWA, https://web.dev/learn/pwa/